

Data Structures

Lecture 12: AVL Tree

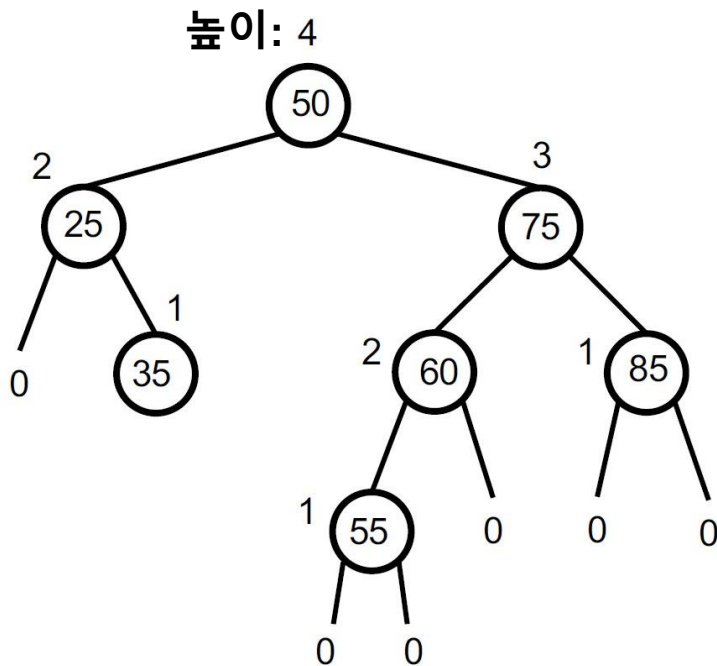
Soobin Um

Apr. 15, 2026

AVL Trees

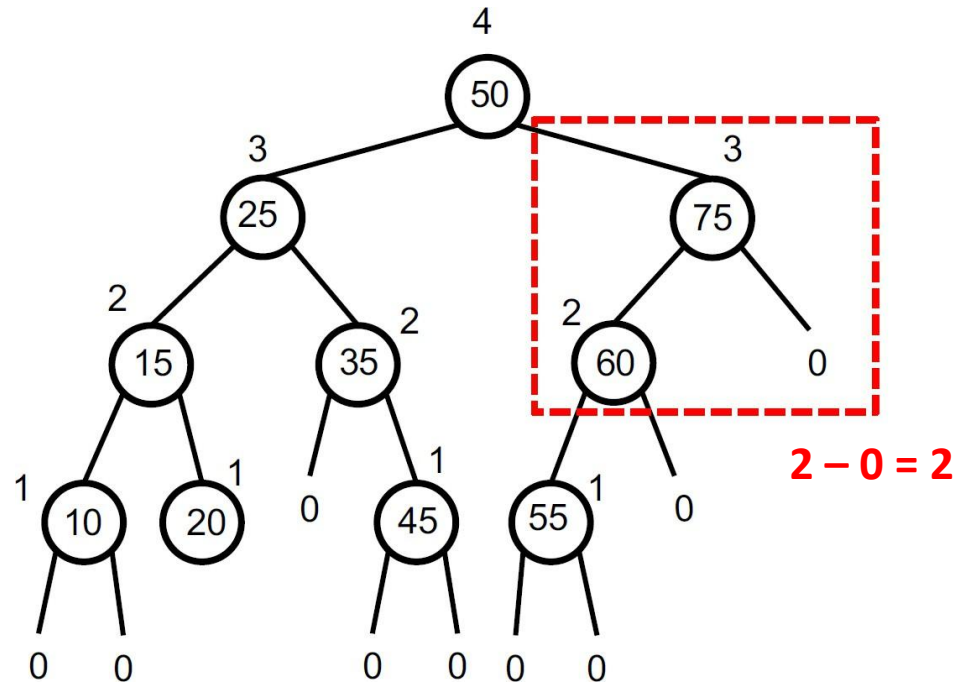
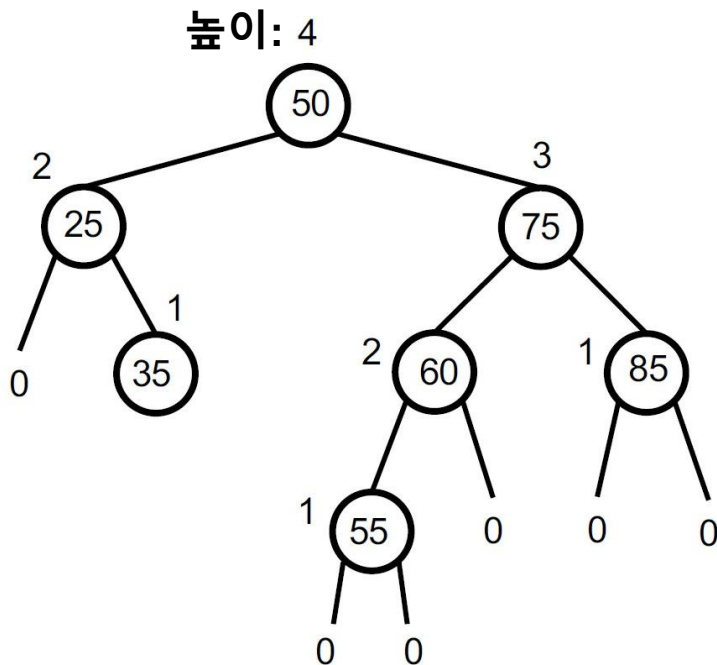
AVL (Adelson-Velsky and Landis) Tree

- 모든 노드에서 왼쪽과 오른쪽 서브트리 간의 **높이 차이가 최대 1을 넘지 않도록 보장**하는 균형 이진 탐색 트리 (**self-balancing BST**)
 - Balance Factor: 왼쪽 서브트리와 오른쪽 서브트리의 높이 차이 (즉, 균형 상태를 수치화한 값)



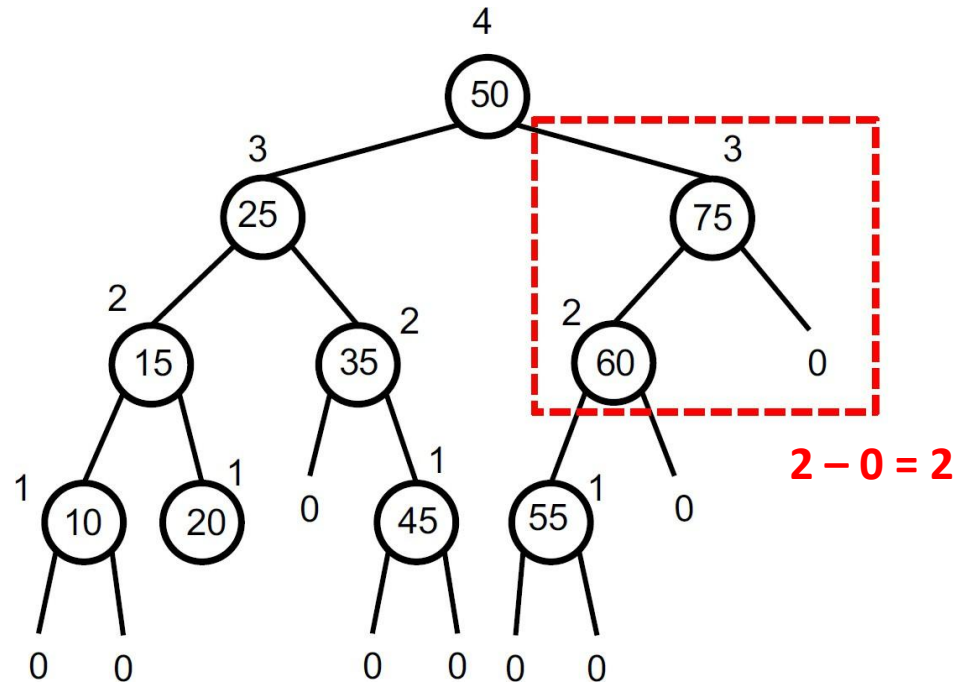
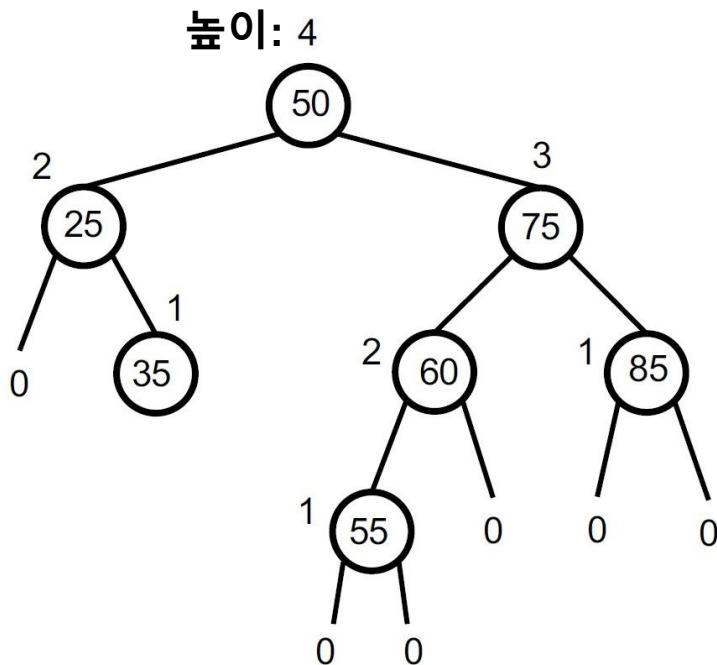
AVL (Adelson-Velsky and Landis) Tree

- 모든 노드에서 왼쪽과 오른쪽 서브트리 간의 **높이 차이가 최대 1을 넘지 않도록 보장**하는 균형 이진 탐색 트리 (**self-balancing BST**)
 - Balance Factor: 왼쪽 서브트리와 오른쪽 서브트리의 높이 차이 (즉, 균형 상태를 수치화한 값)



AVL (Adelson-Velsky and Landis) Tree

- 모든 노드에서 왼쪽과 오른쪽 서브트리 간의 **높이 차이가 최대 1을 넘지 않도록 보장**하는 균형 이진 탐색 트리 (**self-balancing BST**)
 - Balance Factor: 왼쪽 서브트리와 오른쪽 서브트리의 높이 차이 (즉, 균형 상태를 수치화한 값)



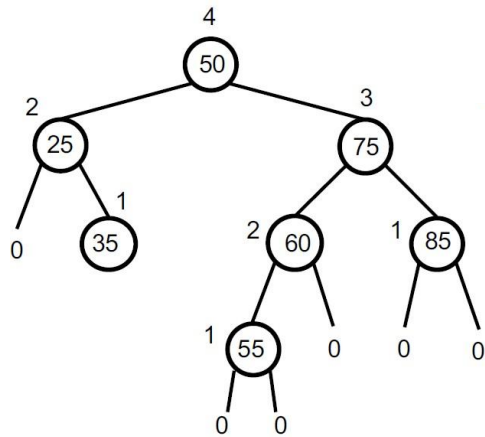
AVL Tree 조건 위반

Insertion on AVL Tree

- Key가 w 인 노드 삽입
 - (1) 일반적인 BST에서의 삽입 연산을 통해 key가 w 인 노드를 삽입
 - (2) 삽입된 w 노드에서부터 위로 올라가며 불균형이 발생하는 첫 번째 노드 z 를 찾기
 - (3) 노드 z 를 루트로 갖는 서브트리 상에서의 “적절한 회전”을 통해, 균형 맞춤

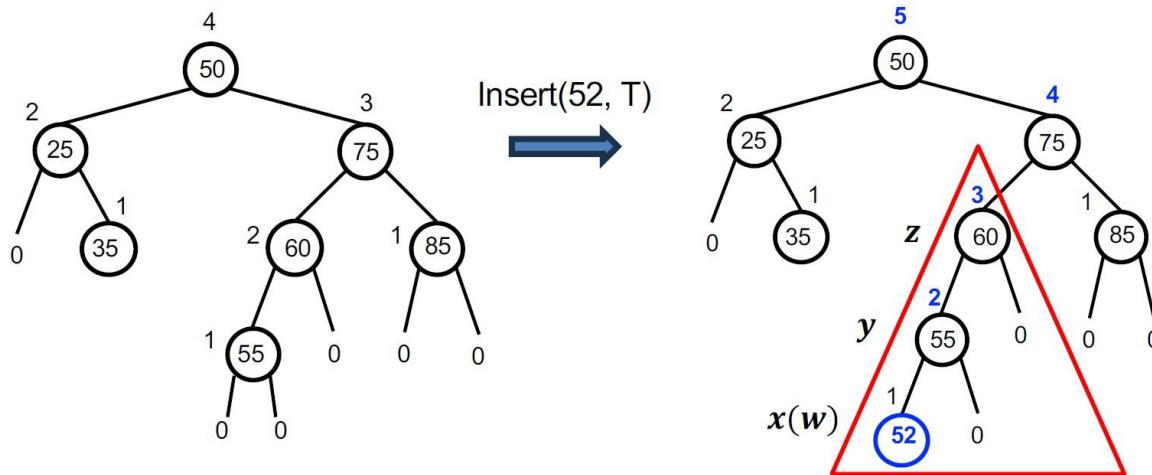
Insertion on AVL Tree

- Key가 w 인 노드 삽입
 - (1) 일반적인 BST에서의 삽입 연산을 통해 key가 w 인 노드를 삽입
 - (2) 삽입된 w 노드에서부터 위로 올라가며 불균형이 발생하는 첫 번째 노드 z 를 찾기
 - (3) 노드 z 를 루트로 갖는 서브트리 상에서의 “적절한 회전”을 통해, 균형 맞춤



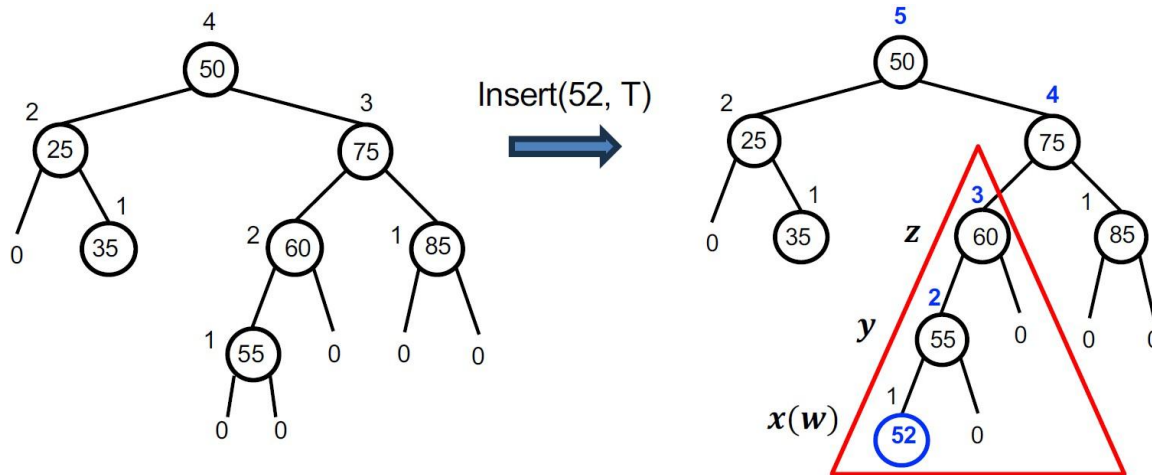
Insertion on AVL Tree

- Key가 w 인 노드 삽입
 - (1) 일반적인 BST에서의 삽입 연산을 통해 key가 w 인 노드를 삽입
 - (2) 삽입된 w 노드에서부터 위로 올라가며 불균형이 발생하는 첫 번째 노드 z 를 찾기
 - (3) 노드 z 를 루트로 갖는 서브트리 상에서의 “적절한 회전”을 통해, 균형 맞춤



Insertion on AVL Tree

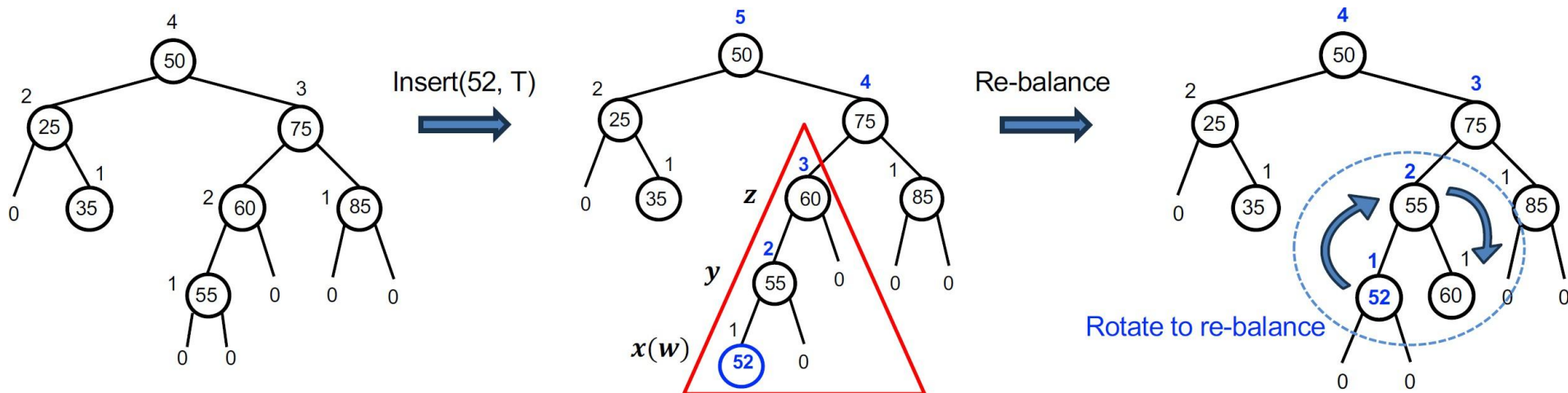
- Key가 w 인 노드 삽입
 - (1) 일반적인 BST에서의 삽입 연산을 통해 key가 w 인 노드를 삽입
 - (2) 삽입된 w 노드에서부터 위로 올라가며 불균형이 발생하는 첫 번째 노드 z 를 찾기
 - (3) 노드 z 를 루트로 갖는 서브트리 상에서의 “적절한 회전”을 통해, 균형 맞춤



새로 삽입된 노드의 조상 중 하나인 노드 z 에서 불균형 발생
($2 - 0 = 2$)

Insertion on AVL Tree

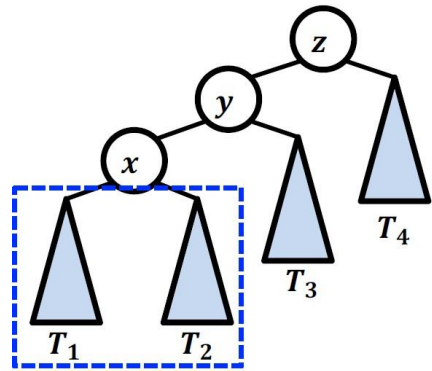
- Key가 w 인 노드 삽입
 - (1) 일반적인 BST에서의 삽입 연산을 통해 key가 w 인 노드를 삽입
 - (2) 삽입된 w 노드에서부터 위로 올라가며 불균형이 발생하는 첫 번째 노드 z 를 찾기
 - (3) 노드 z 를 루트로 갖는 서브트리 상에서의 “적절한 회전”을 통해, 균형 맞춤



새로 삽입된 노드의 조상 중 하나인 노드 z 에서 불균형 발생
(2 - 0 = 2)

Insertion on AVL Tree

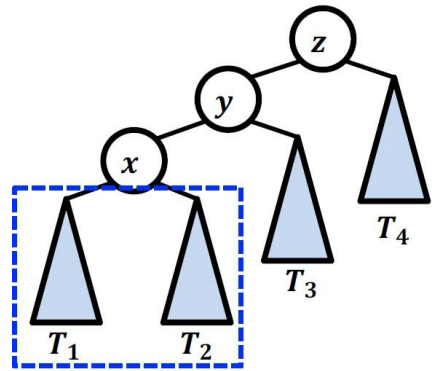
- (노드 z 에서 발생하는) 4가지 불균형 케이스
 - (Case 1) LL: 노드 z 의 왼쪽 자식의 왼쪽 서브트리 내 삽입



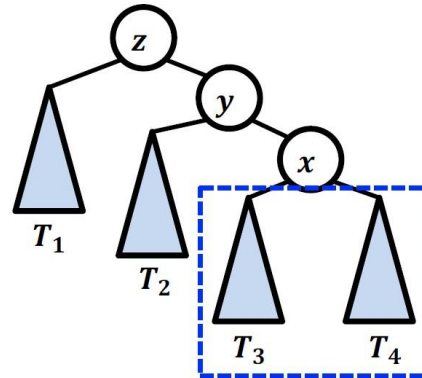
Case 1: LL

Insertion on AVL Tree

- (노드 z 에서 발생하는) 4가지 불균형 케이스
 - (Case 1) LL: 노드 z 의 왼쪽 자식의 왼쪽 서브트리 내 삽입
 - (Case 2) RR: 노드 z 의 오른쪽 자식의 오른쪽 서브트리 내 삽입



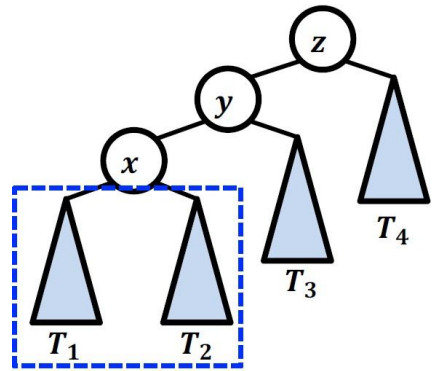
Case 1: LL



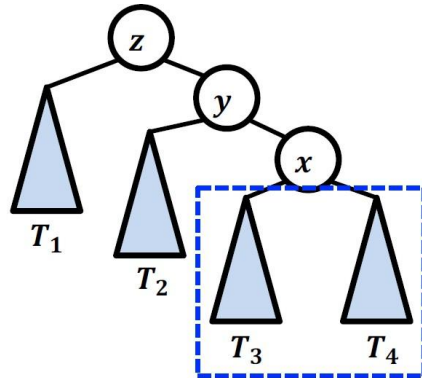
Case 2: RR

Insertion on AVL Tree

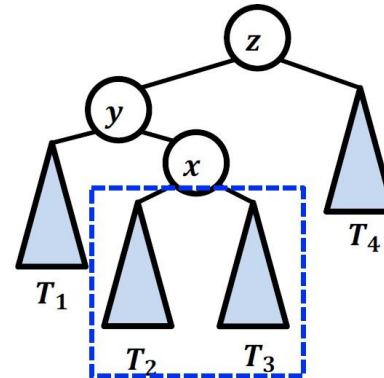
- (노드 z에서 발생하는) 4가지 불균형 케이스
 - (Case 1) LL: 노드 z의 왼쪽 자식의 왼쪽 서브트리 내 삽입
 - (Case 2) RR: 노드 z의 오른쪽 자식의 오른쪽 서브트리 내 삽입
 - (Case 3) LR: 노드 z의 왼쪽 자식의 오른쪽 서브트리 내 삽입



Case 1: LL



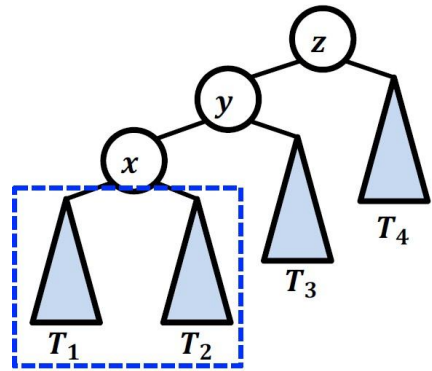
Case 2: RR



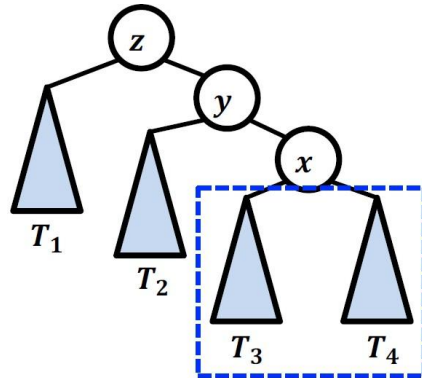
Case 3: LR

Insertion on AVL Tree

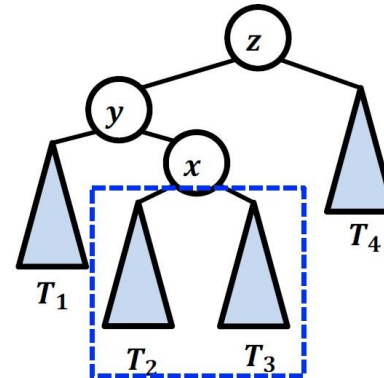
- (노드 z에서 발생하는) 4가지 불균형 케이스
 - (Case 1) LL: 노드 z의 왼쪽 자식의 왼쪽 서브트리 내 삽입
 - (Case 2) RR: 노드 z의 오른쪽 자식의 오른쪽 서브트리 내 삽입
 - (Case 3) LR: 노드 z의 왼쪽 자식의 오른쪽 서브트리 내 삽입
 - (Case 4) RL: 노드 z의 오른쪽 자식의 왼쪽 서브트리 내 삽입



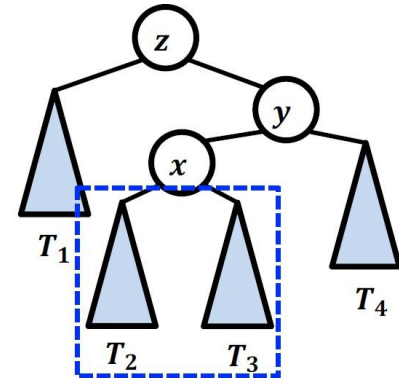
Case 1: LL



Case 2: RR



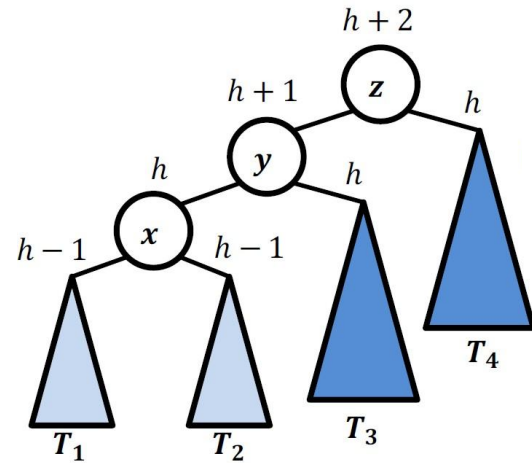
Case 3: LR



Case 4: RL

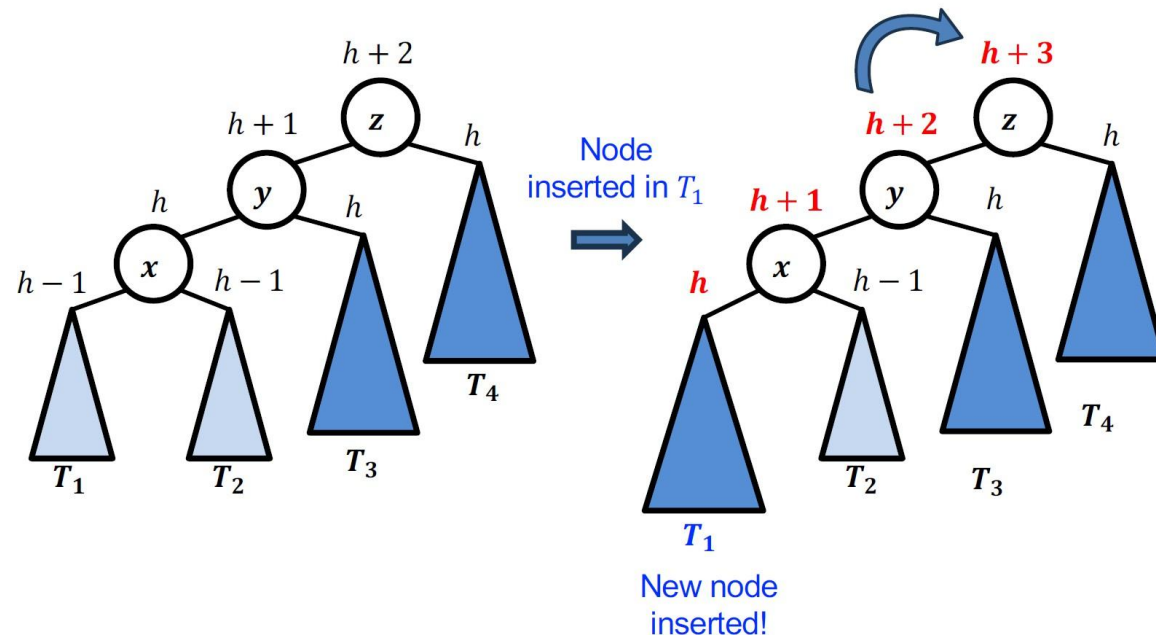
Insertion on AVL Tree (Case 1)

- **(Case 1) LL:** 노드 z 의 왼쪽 자식의 왼쪽 서브트리 내 삽입
 - 노드 z 에서의 “Right Rotation”을 통해 균형 맞춤



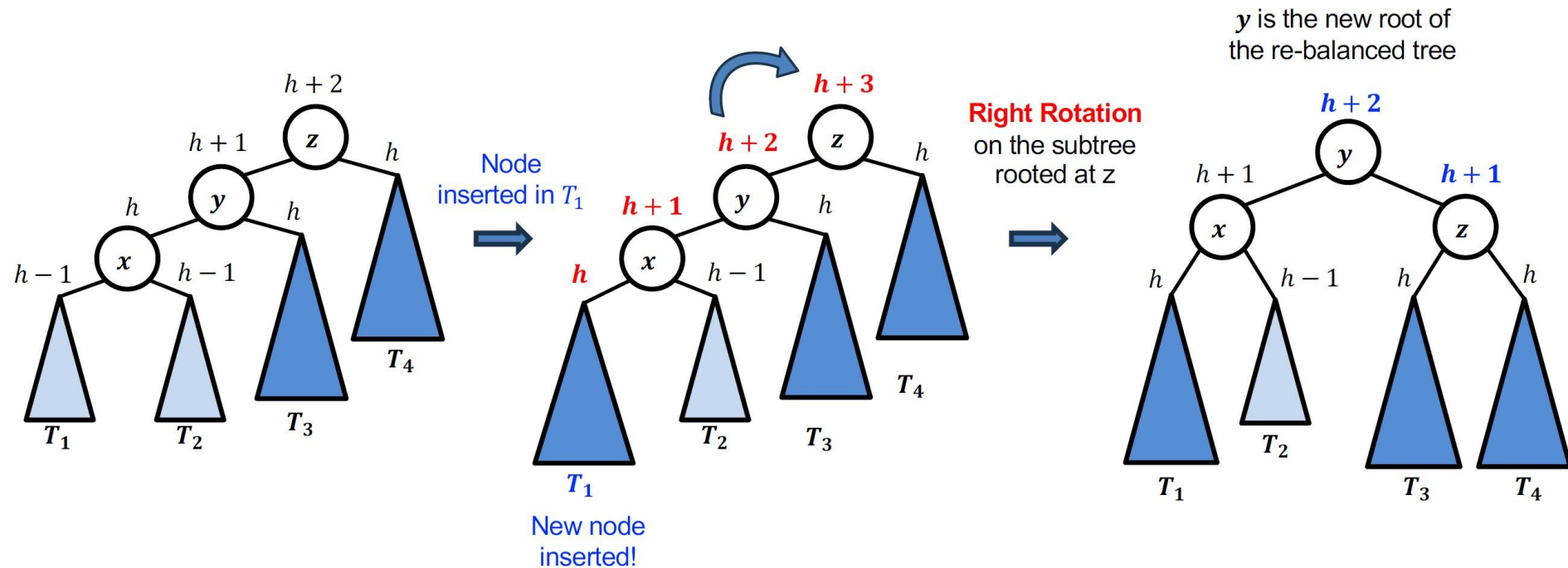
Insertion on AVL Tree (Case 1)

- **(Case 1) LL:** 노드 z 의 왼쪽 자식의 왼쪽 서브트리 내 삽입
 - 노드 z 에서의 “Right Rotation”을 통해 균형 맞춤



Insertion on AVL Tree (Case 1)

- **(Case 1) LL:** 노드 z 의 왼쪽 자식의 왼쪽 서브트리 내 삽입
 - 노드 z 에서의 “Right Rotation”을 통해 균형 맞춤

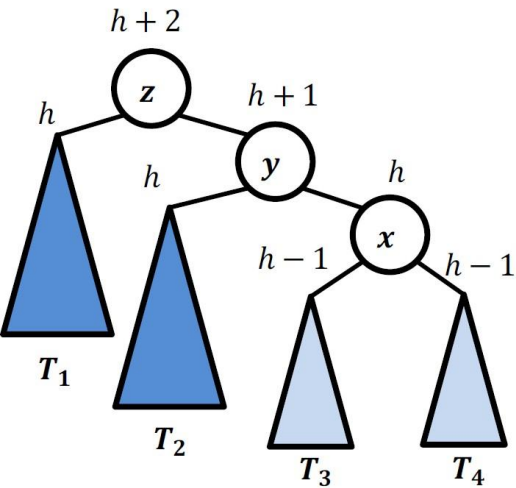


Right Rotation:

노드 z 가 y 의 오른쪽 자식 노드가 되며,
 y 의 기존 오른쪽 자식 노드는 z 의 왼쪽 자식 노드가 됨

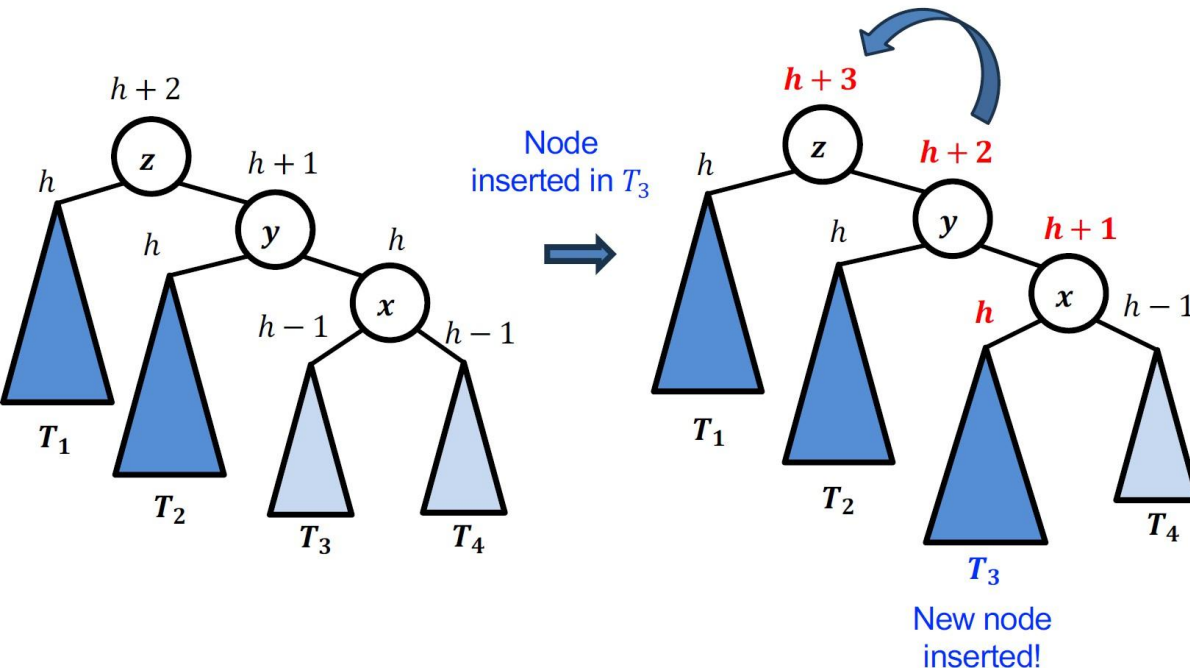
Insertion on AVL Tree (Case 2)

- **(Case 2) RR:** 노드 z 의 오른쪽 자식의 오른쪽 서브트리 내 삽입
 - 노드 z 에서의 “Left Rotation”을 통해 균형 맞춤



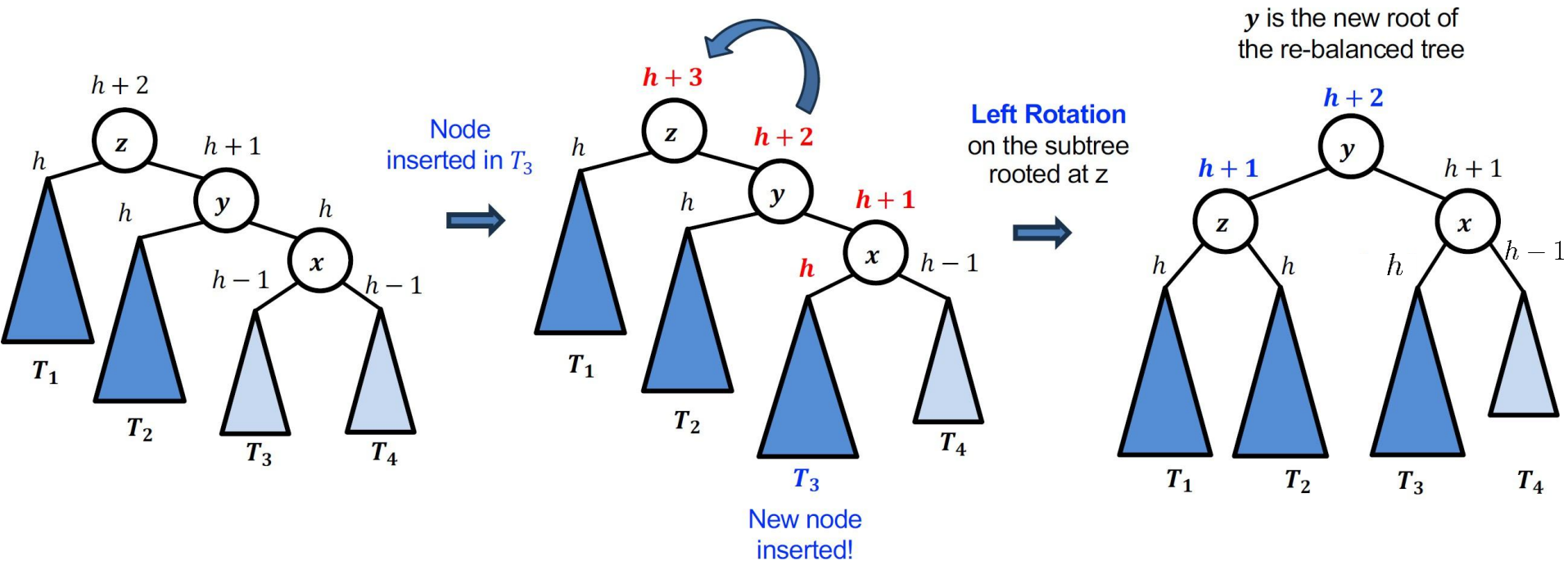
Insertion on AVL Tree (Case 2)

- **(Case 2) RR:** 노드 z 의 오른쪽 자식의 오른쪽 서브트리 내 삽입
 - 노드 z 에서의 “Left Rotation”을 통해 균형 맞춤



Insertion on AVL Tree (Case 2)

- **(Case 2) RR:** 노드 z 의 오른쪽 자식의 오른쪽 서브트리 내 삽입
 - 노드 z 에서의 “Left Rotation”을 통해 균형 맞춤

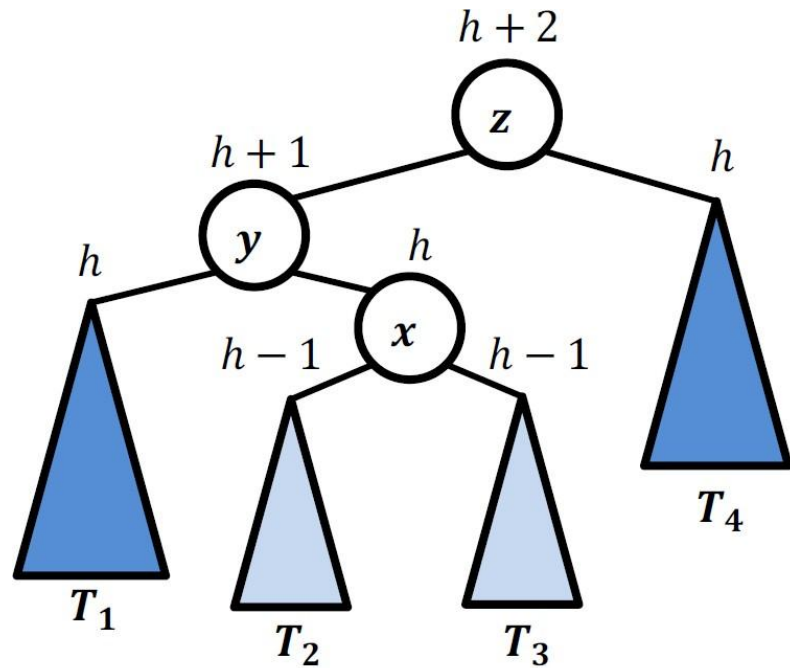


Left Rotation:

노드 z 가 y 의 **왼쪽** 자식 노드가 되며,
 y 의 기존 왼쪽 자식 노드는 z 의 오른쪽 자식 노드가 됨

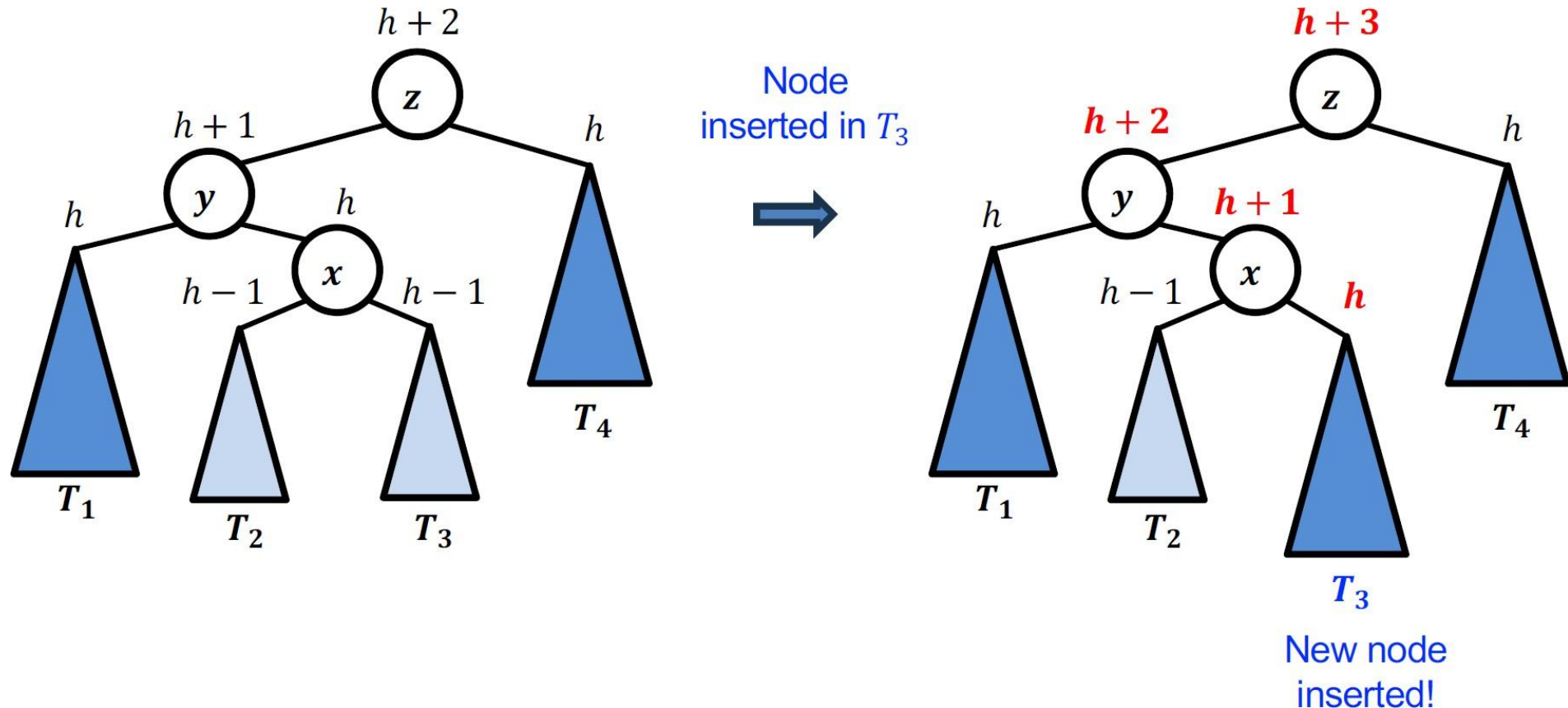
Insertion on AVL Tree (Case 3)

- **(Case 3) LR:** 노드 z 의 왼쪽 자식의 오른쪽 서브트리 내 삽입



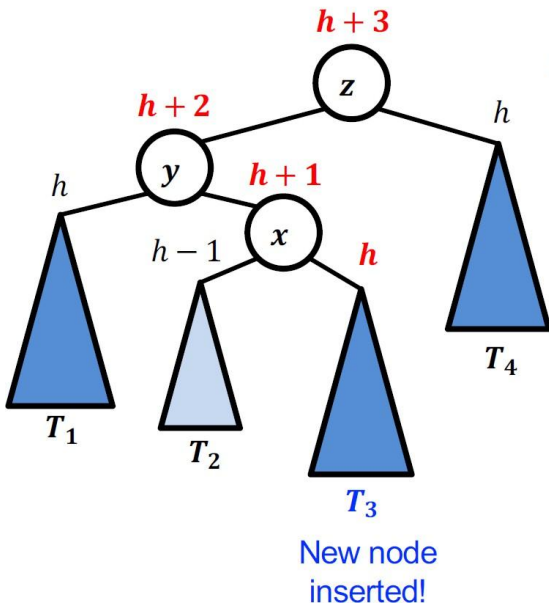
Insertion on AVL Tree (Case 3)

- (Case 3) LR: 노드 z 의 왼쪽 자식의 오른쪽 서브트리 내 삽입



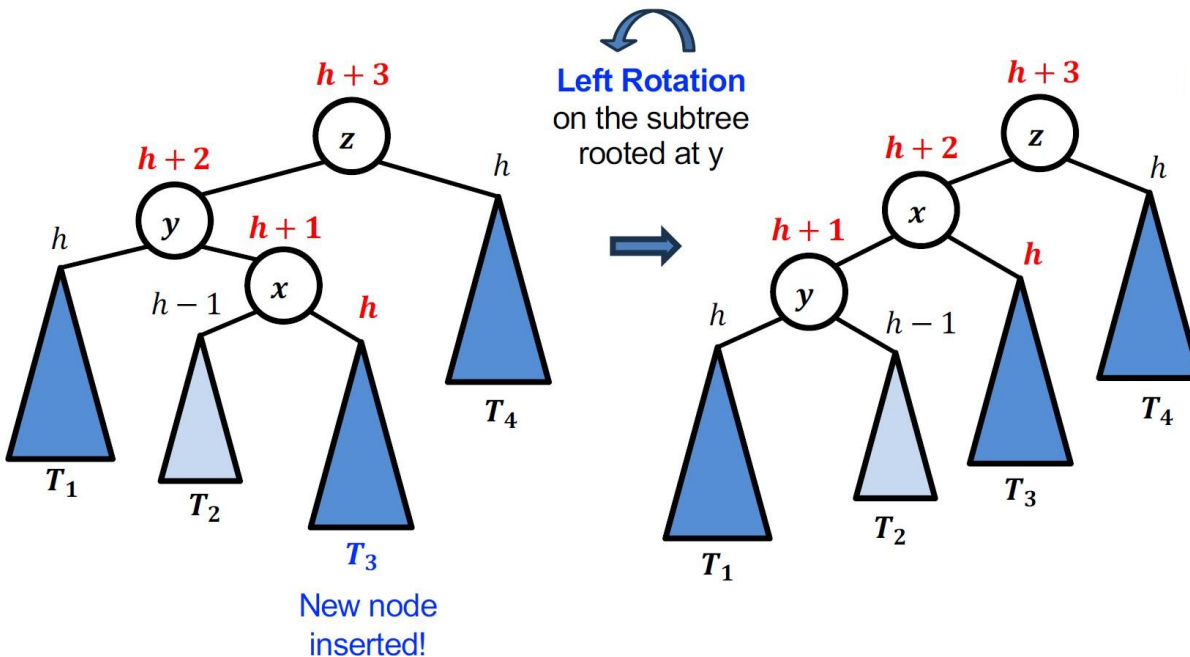
Insertion on AVL Tree (Case 3)

- **(Case 3) LR:** 노드 z 의 왼쪽 자식의 오른쪽 서브트리 내 삽입
 - 노드 y 에서의 “Left Rotation” 이후, z 에서의 “Right Rotation”을 통해 균형 맞춤



Insertion on AVL Tree (Case 3)

- **(Case 3) LR:** 노드 z 의 왼쪽 자식의 오른쪽 서브트리 내 삽입
 - 노드 y 에서의 “Left Rotation” 이후, z 에서의 “Right Rotation”을 통해 균형 맞춤

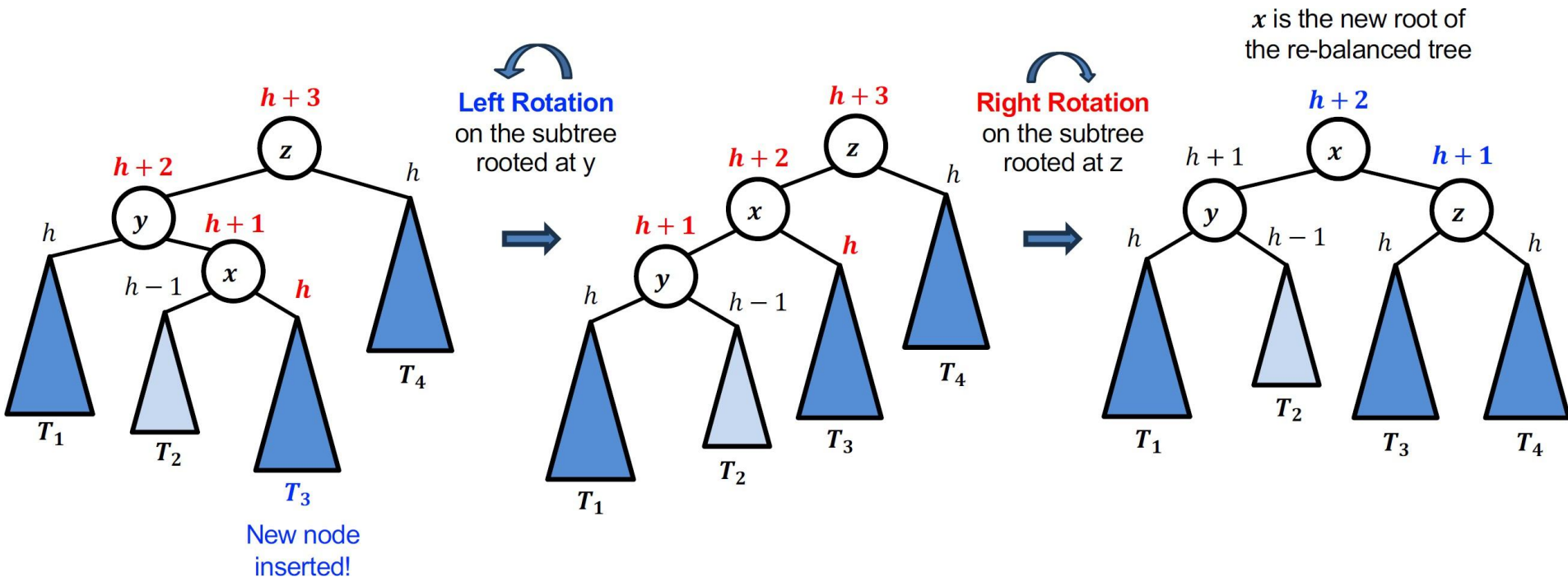


Left Rotation:

노드 y 가 x 의 왼쪽 자식 노드가 되며,
 x 의 기존 왼쪽 자식 노드는 y 의 오른쪽 자식 노드가 됨

Insertion on AVL Tree (Case 3)

- **(Case 3) LR:** 노드 z 의 왼쪽 자식의 오른쪽 서브트리 내 삽입
 - 노드 y 에서의 “Left Rotation” 이후, z 에서의 “Right Rotation”을 통해 균형 맞춤



Left Rotation:

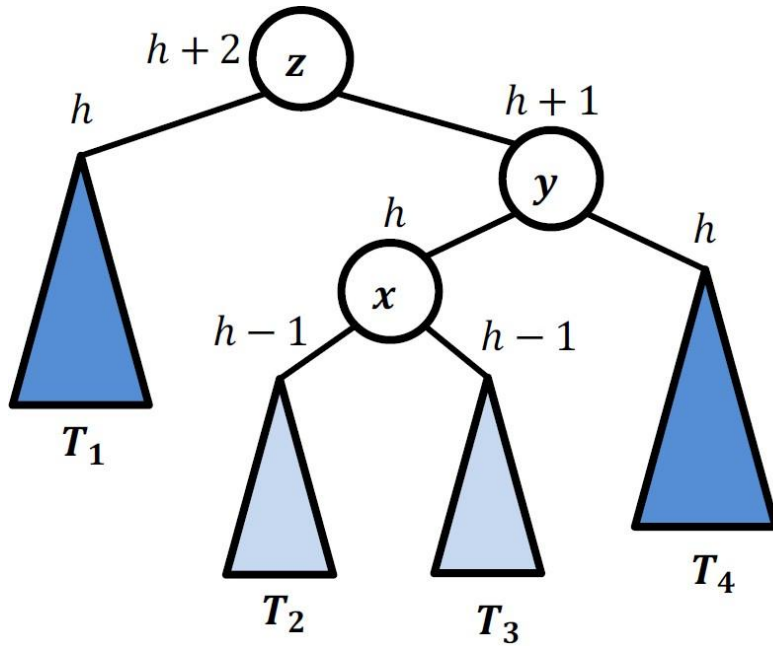
노드 y 가 x 의 왼쪽 자식 노드가 되며,
 x 의 기존 왼쪽 자식 노드는 y 의 오른쪽 자식 노드가 됨

Right Rotation:

노드 z 가 x 의 오른쪽 자식 노드가 되며,
 x 의 기존 오른쪽 자식 노드는 z 의 왼쪽 자식 노드가 됨

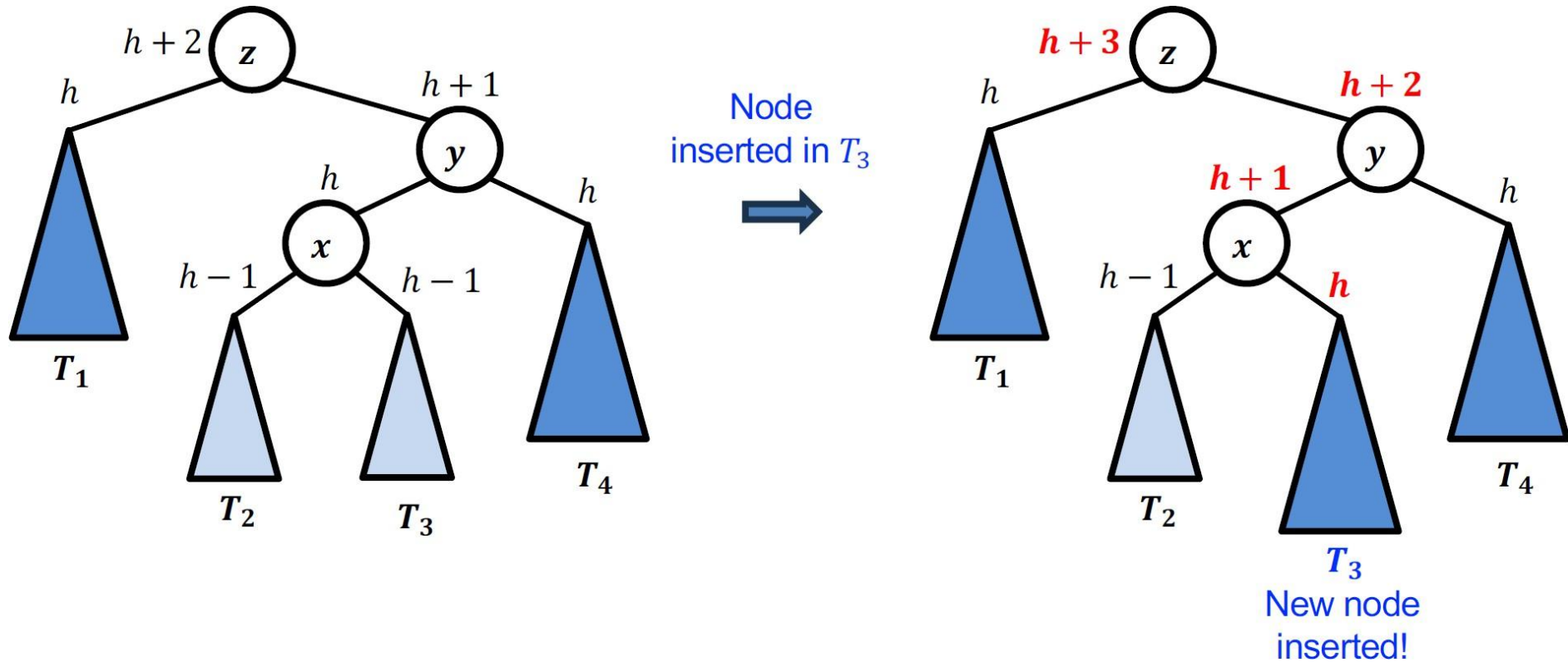
Insertion on AVL Tree (Case 4)

- (Case 4) RL: 노드 z 의 오른쪽 자식의 왼쪽 서브트리 내 삽입



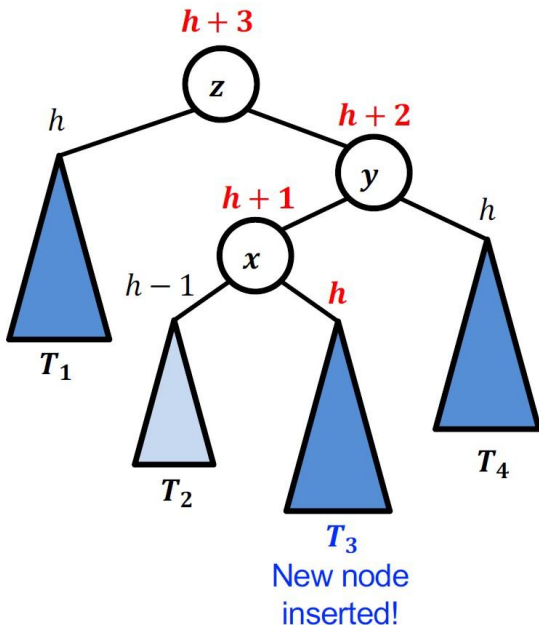
Insertion on AVL Tree (Case 4)

- (Case 4) RL: 노드 z 의 오른쪽 자식의 왼쪽 서브트리 내 삽입



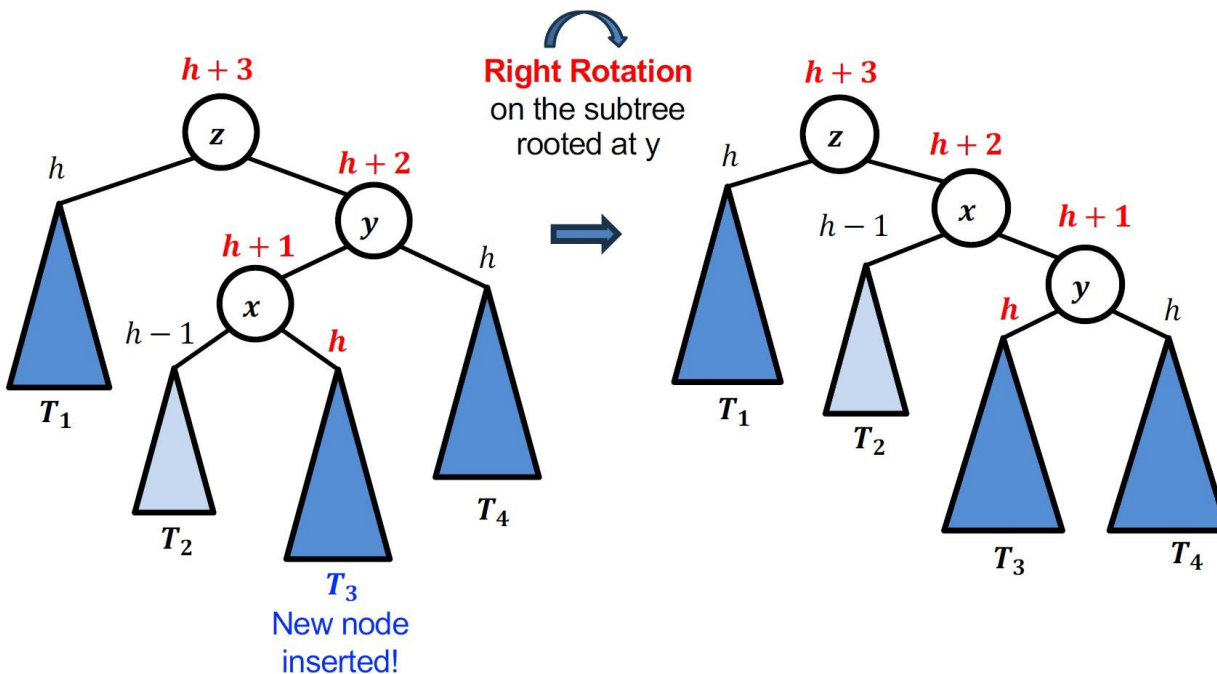
Insertion on AVL Tree (Case 4)

- **(Case 4) RL:** 노드 z 의 오른쪽 자식의 왼쪽 서브트리 내 삽입
 - 노드 y 에서의 “Right Rotation” 이후, z 에서의 “Left Rotation”을 통해 균형 맞춤



Insertion on AVL Tree (Case 4)

- **(Case 4) RL:** 노드 z 의 오른쪽 자식의 왼쪽 서브트리 내 삽입
 - 노드 y 에서의 “Right Rotation” 이후, z 에서의 “Left Rotation”을 통해 균형 맞춤

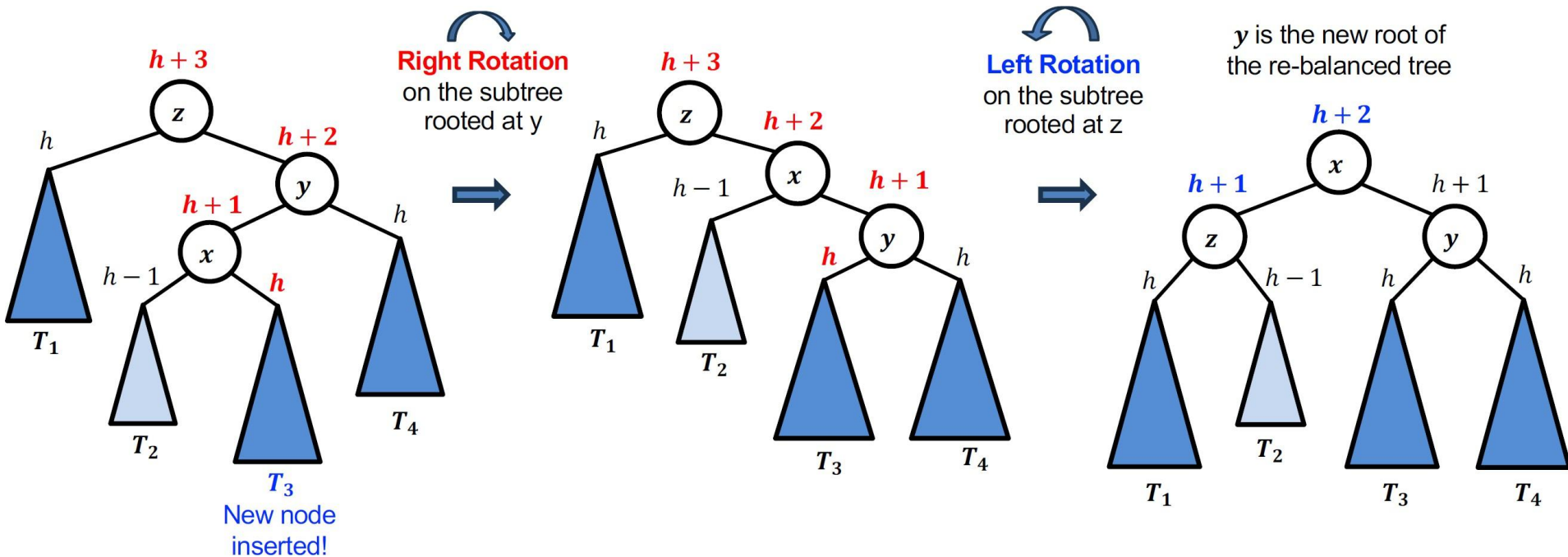


Right Rotation:

노드 y 가 x 의 오른쪽 자식 노드가 되며,
 x 의 기존 오른쪽 자식 노드는 y 의 왼쪽 자식 노드가 됨

Insertion on AVL Tree (Case 4)

- **(Case 4) RL:** 노드 z 의 오른쪽 자식의 왼쪽 서브트리 내 삽입
 - 노드 y 에서의 “Right Rotation” 이후, z 에서의 “Left Rotation”을 통해 균형 맞춤



Right Rotation:

노드 y 가 x 의 **오른쪽** 자식 노드가 되며,
 x 의 기존 오른쪽 자식 노드는 y 의 왼쪽 자식 노드가 됨

Left Rotation:

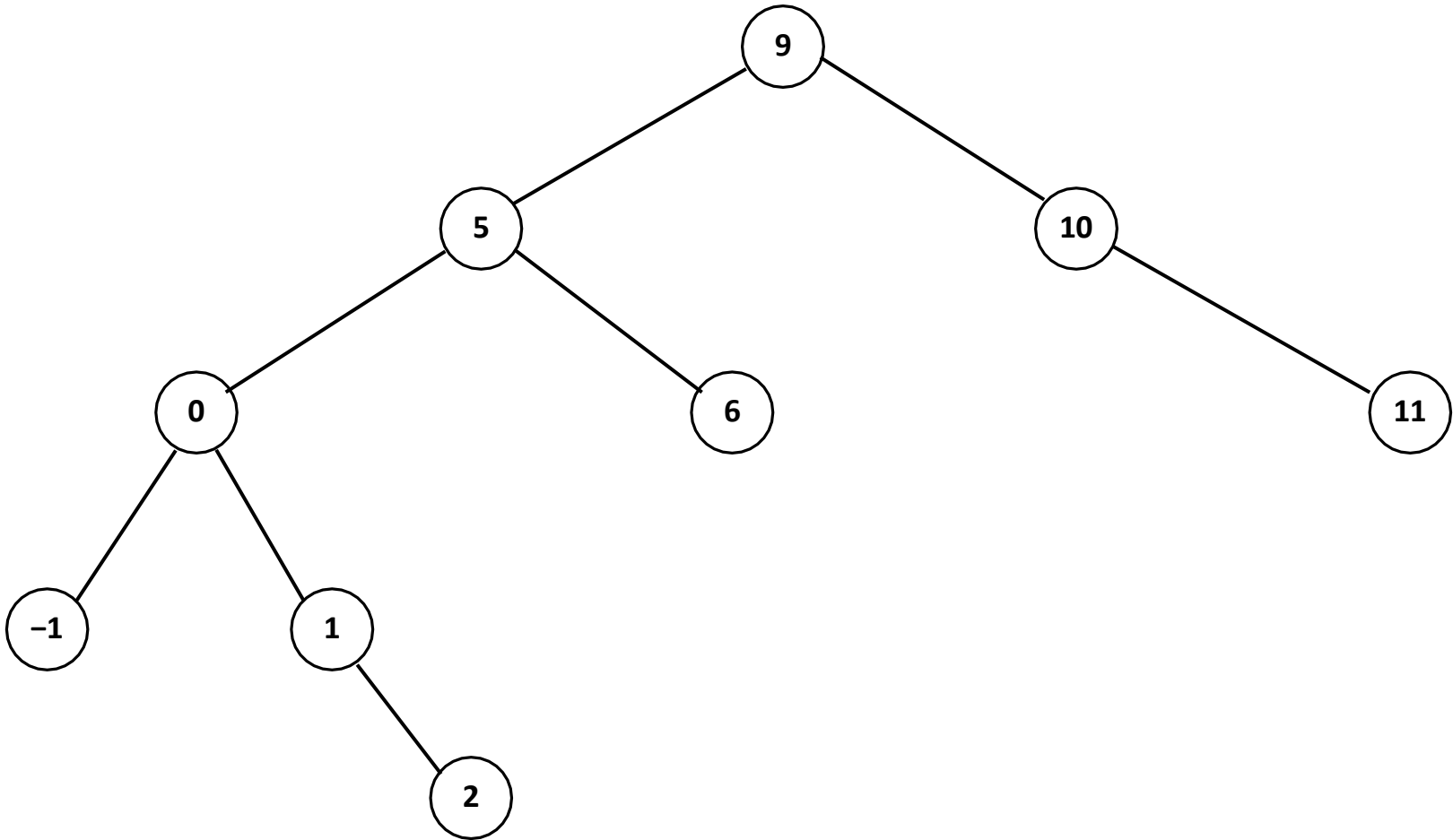
노드 z 가 x 의 **왼쪽** 자식 노드가 되며,
 x 의 기존 왼쪽 자식 노드는 z 의 오른쪽 자식 노드가 됨

Exercise

- AVL 트리 그리기
 - 삽입: <9, 5, 10, 0, 6, 11, -1, 1, 2>

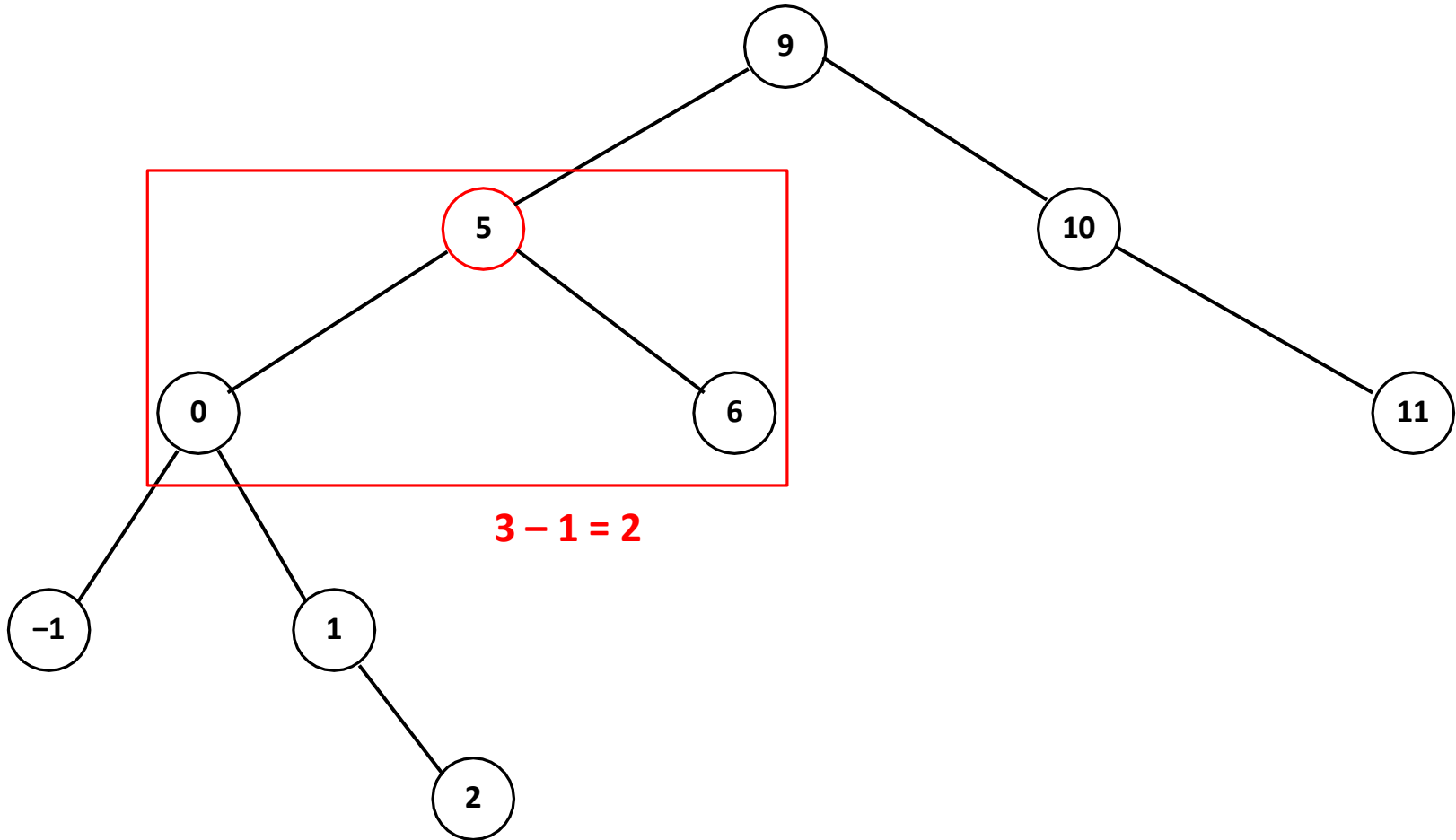
Exercise

- 삽입: <9, 5, 10, 0, 6, 11, -1, 1, 2>
 - (1) 이진탐색트리 (BST) 구성



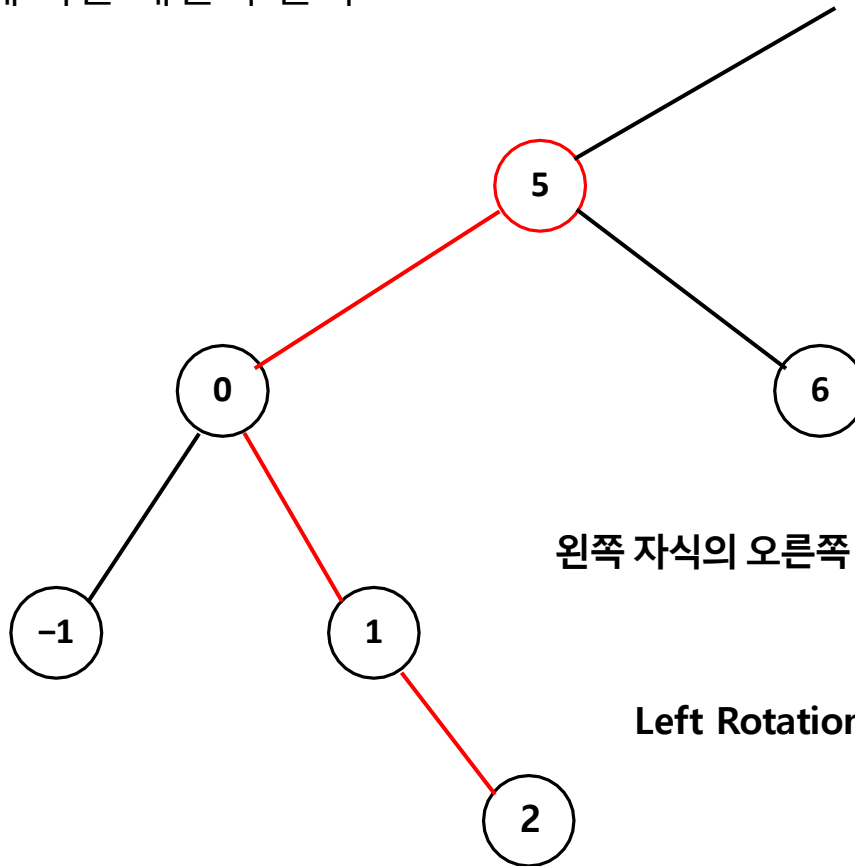
Exercise

- 삽입: <9, 5, 10, 0, 6, 11, -1, 1, 2>
 - (2) 불균형이 발생하는 첫 번째 노드 찾기



Exercise

- 삽입: <9, 5, 10, 0, 6, 11, -1, 1, 2>
 - (3) Case에 따른 해결책 선택



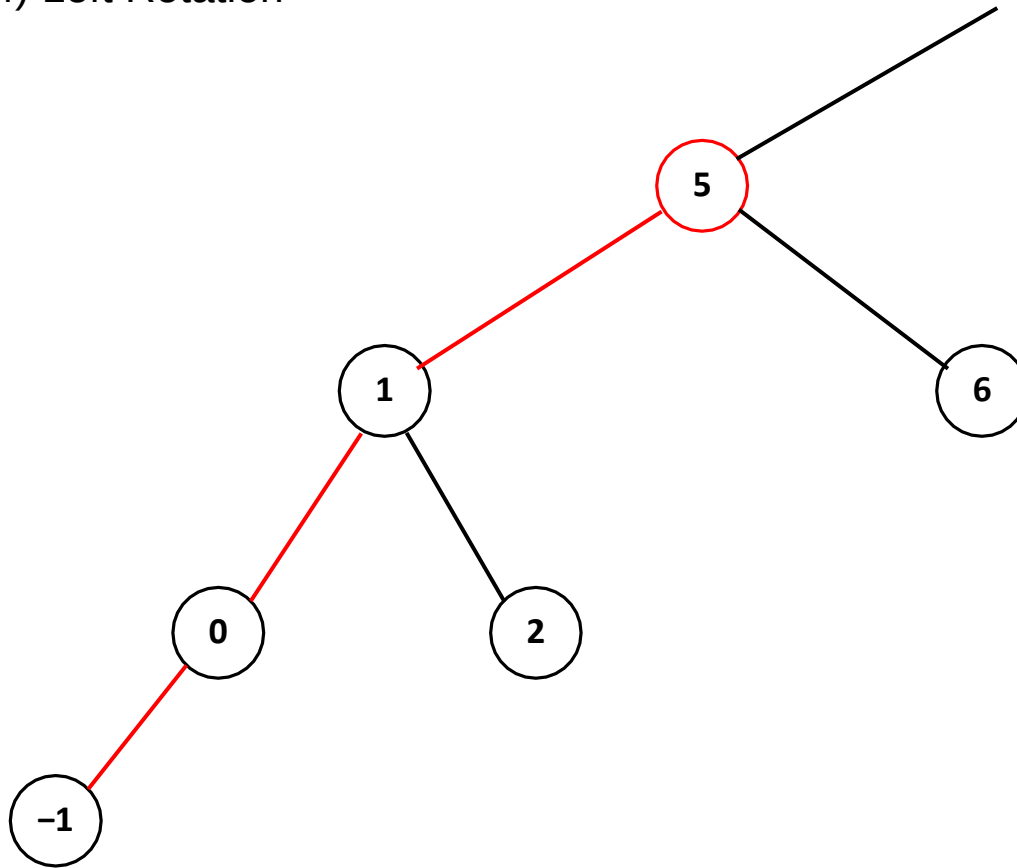
왼쪽 자식의 오른쪽 서브트리 (Case 3 – LR)



Left Rotation 후 Right Rotation 수행

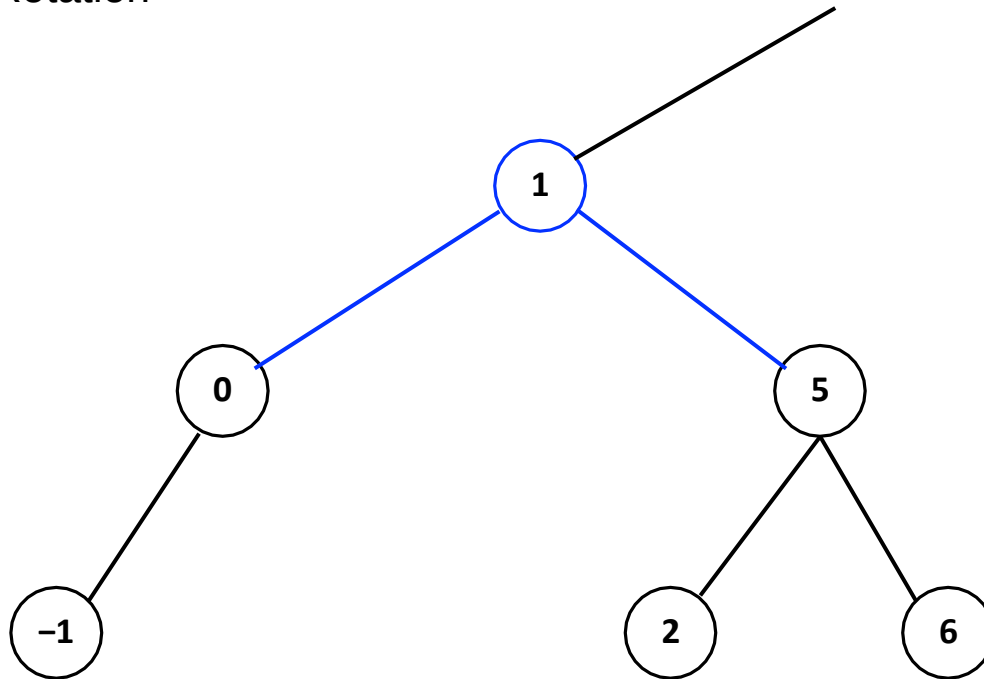
Exercise

- 삽입: <9, 5, 10, 0, 6, 11, -1, 1, 2>
 - (4) Left Rotation



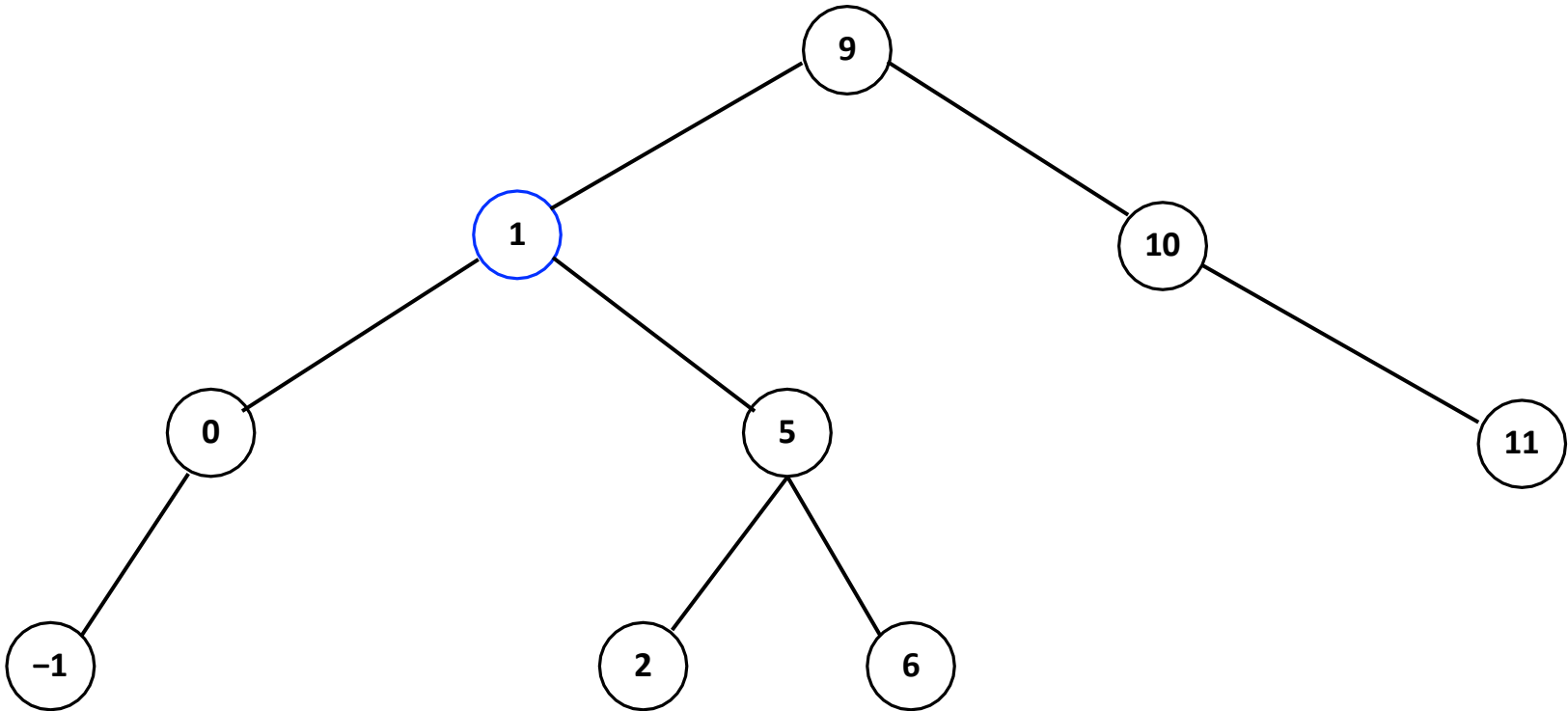
Exercise

- 삽입: <9, 5, 10, 0, 6, 11, -1, 1, 2>
 - (5) Right Rotation



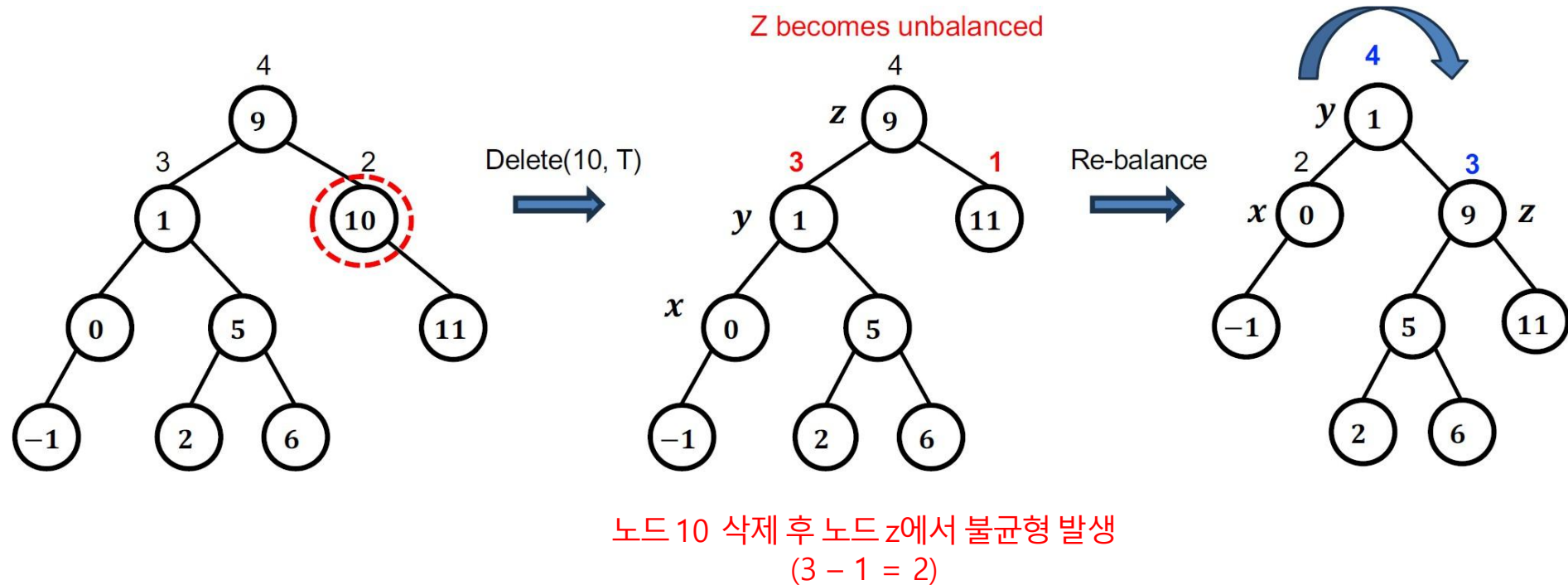
Exercise

- 삽입: $\langle 9, 5, 10, 0, 6, 11, -1, 1, 2 \rangle$
 - (6) 완성



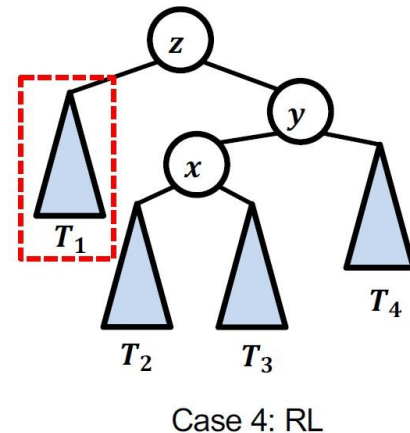
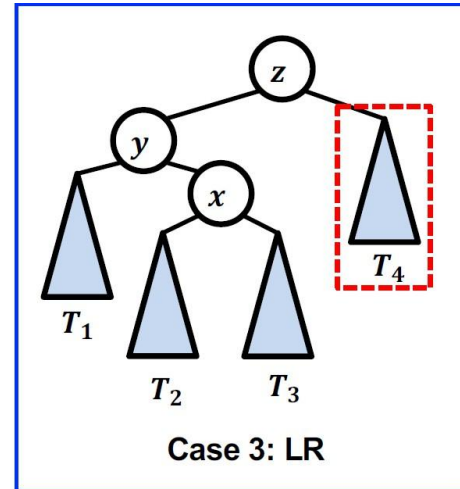
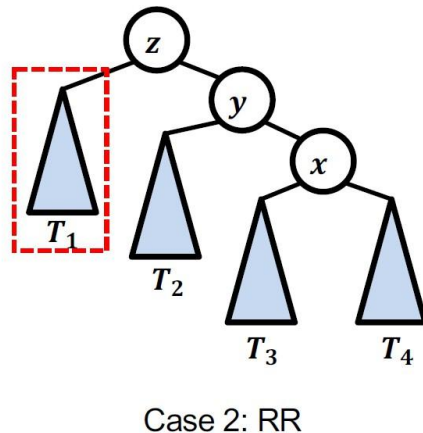
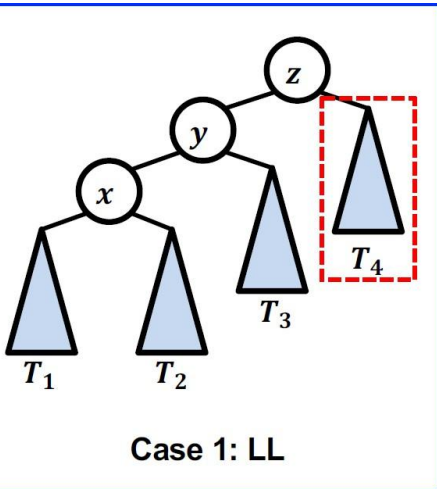
Deletion on AVL Tree

- Key가 w 인 노드 삭제
 - (1) 일반적인 BST에서의 삭제 연산을 통해 key가 w 인 노드를 삭제
 - (2) Key가 w 인 노드 삭제로 인해 불균형이 발생하는 노드 z 를 찾기
 - (3) 노드 z 를 루트로 갖는 서브트리 상에서의 “적절한 회전”을 통해, 균형 맞춤



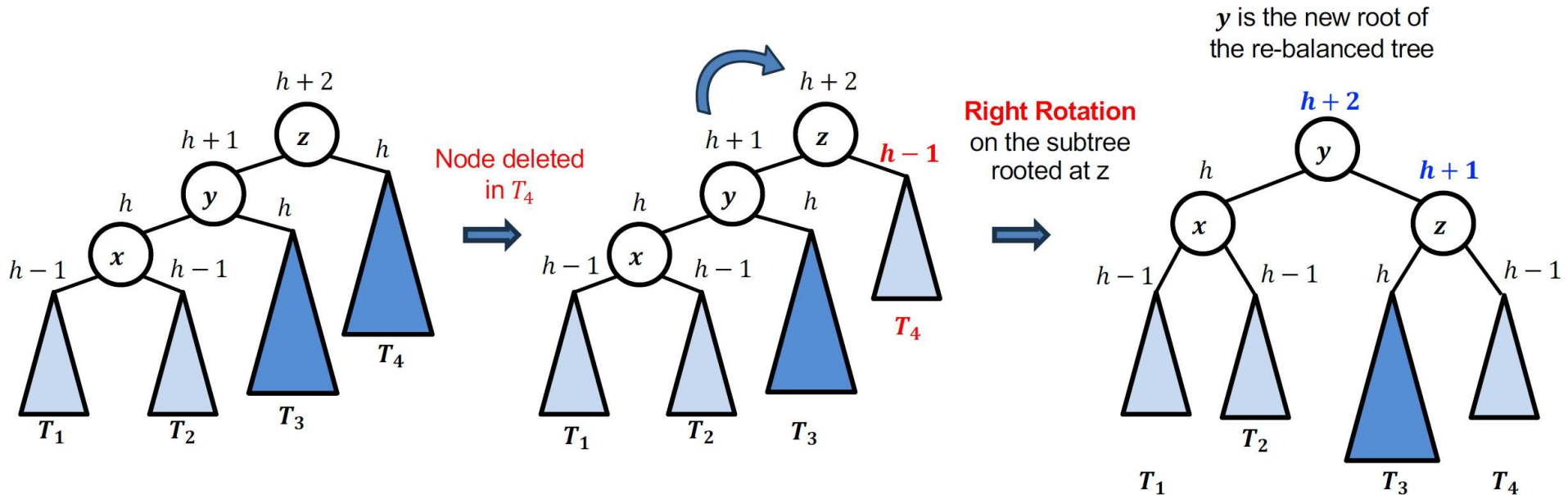
Deletion on AVL Tree

- (노드 z에서 발생하는) 4가지 불균형 케이스
 - 노드 z의 오른쪽 서브트리(= T_4)에서 삭제가 발생한 경우: (Case 1), (Case 3)
 - 노드 z의 왼쪽 서브트리(= T_1)에서 삭제가 발생한 경우: (Case 2), (Case 4)



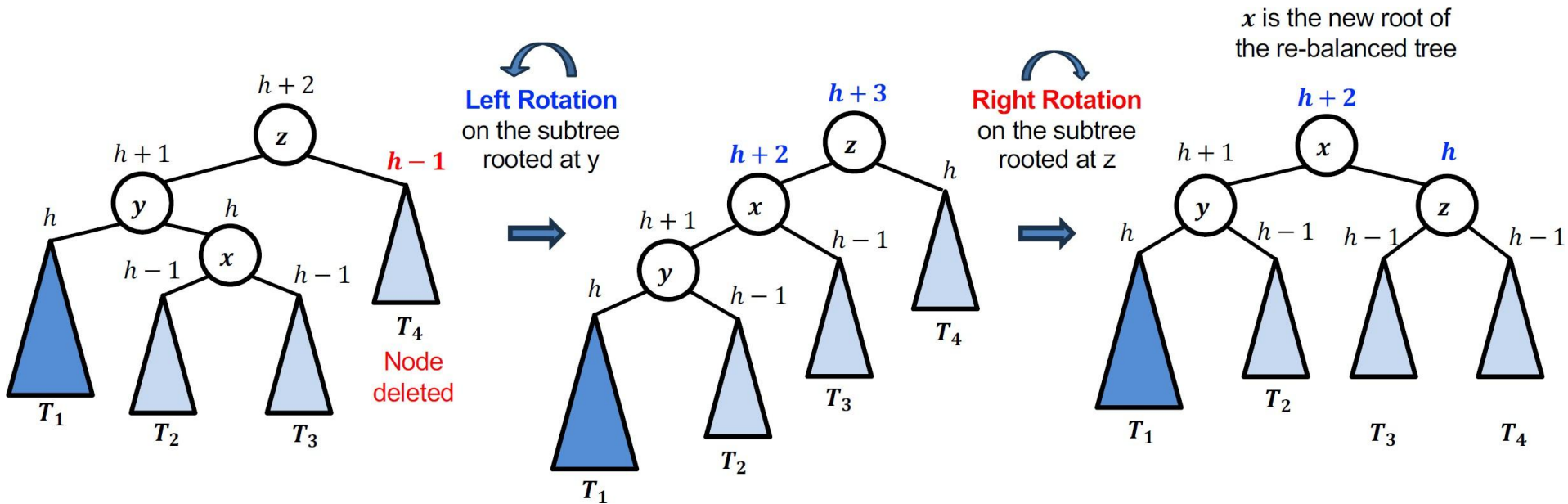
Deletion on AVL Tree (Case 1)

- T_4 에서의 노드 삭제: (Case 1) LL
 - 노드 z 에서의 “Right Rotation”을 통해 균형 맞춤



Deletion on AVL Tree (Case 3)

- **T4**에서의 노드 삭제: (Case 3) LR
 - 노드 y 에서의 “Left Rotation” 이후, z 에서의 “Right Rotation”을 통해 균형 맞춤



Summary

- **Pros**

- 스스로 균형(self-balancing)을 유지함으로써, **트리의 높이를 항상 $O(\log n)$ 으로 보장함**
- 탐색, 삽입, 삭제 연산 모두에서 효율적인 수행 시간을 보장

- **Cons**

- 구현 및 관리가 복잡함
- 일부 연산에서 **상수 시간 오버헤드가 큼**
 - 삽입/삭제 연산 시, **balance factor 갱신**, **회전 연산** 등 추가적인 상수 시간 작업이 반복적으로 발생
 - Red-Black Tree에 비해 삽입 또는 삭제 연산에서 발생하는 비용이 더 큼

Look ahead

중간고사

중간고사 관련 공지

- 일시: 4/20(월) 12:00~13:00
- 장소: 미래관 611호, 629호
→ 인원별 강의실 배정은 eCampus 공지 참조.

반드시 본인이 배정된 강의실에서 시험 치러야 함.

- 시험범위: LS01-LS12
 - 시험방식: 지필고사(**시험 중 전자기기 사용 금지**)
- ※ 한국어/영어 시험지 선택 가능